adherence to the approved Dockerfile, synchronize any changes throughout the organization, and understand who is responsible for what.

Cloud Native Buildpacks provide a different approach, focusing on consistency, security, performance, and governance. As a developer, you get a tool that automatically builds a production-ready OCI image from your application source code without having to write a Dockerfile. As an operator, you get a tool that defines, controls, and secures application artifacts within the entire organization.

Ultimately, the decision to use a Dockerfile or a tool like Buildpacks depends on your organization and requirements. Both approaches are valid and are used in production. In general, my recommendation is to use Buildpacks unless it makes sense not to.

NOTE Another option for packaging Java applications as container images without writing a Dockerfile is using Jib, a Gradle and Maven plugin developed by Google (https://github.com/GoogleContainerTools/jib).

In the next section and the rest of the book, we'll use Cloud Native Buildpacks rather than Dockerfiles. It was important for me to show you how a Dockerfile works, because it makes understanding the container image features and layers easier. Furthermore, I wanted to show you how to write a basic Dockerfile for containerizing a Spring Boot application to highlight what's needed and illustrate the execution of the application JAR from within the container. Finally, it will be easier for you to debug containers when something goes wrong, even if they're automatically generated by Buildpacks, because you now know how to build images from scratch. If you'd like to learn more about Dockerfiles for Spring Boot applications, I recommend you look at the official documentation (https://spring.io/projects/spring-boot).